# CI/CD Pipeline Implementation for a Scientific Calculator

Software Production Engineering Mini Project Report

Keyur Sanjaykumar Padiya
MT2025065

March 7, 2026

# 1 Introduction

This project demonstrates the implementation of a **Continuous Integration and Continuous Deployment (CI/CD)** pipeline for a **Scientific Calculator** application developed using **Python**.

The calculator provides mathematical operations such as addition, subtraction, multiplication, division, factorial, natural logarithm, power, and square root. The application runs through a menu-driven command line interface that allows users to perform calculations interactively.

The objective of this project is to automate the process of building, testing, containerizing, and deploying the application whenever new code changes are pushed to the repository. Automation ensures that application updates are tested and deployed consistently with minimal manual effort.

The project uses multiple **DevOps tools** to achieve this workflow. **GitHub** is used for version control and source code management. **Jenkins** automates the **CI/CD pipeline** and executes each stage of the workflow. Automated testing is performed using the **Pytest** framework to verify the correctness of calculator operations. **Docker** is used to containerize the application so that it runs consistently across different environments. The container image is stored on **DockerHub**, which acts as a registry for distributing Docker images. Finally, **Ansible** is used to automate deployment by pulling the latest image and running the container.

The overall workflow of the system is shown below:

**Developer → GitHub → Jenkins → Pytest Testing → Docker Build → DockerHub → Ansible Deployment**

## 1.1 Tools Used

| **Tool** | **Purpose** |
|---|---|
| Git & GitHub | Git is used for version control and GitHub is used as a remote repository to manage source code. |
| Pytest | Used for automated unit testing of calculator operations. |
| Jenkins | Used to automate the CI/CD pipeline and execute build, test, and deployment stages. |
| Docker | Used to containerize the calculator application and provide a consistent runtime environment. |
| DockerHub | Used as a container registry to store and distribute Docker images. |
| Ansible | Used to automate deployment by pulling the latest Docker image and running the container. |

Table 1: Tools Used in the DevOps Pipeline

# 2 Source Code Management

GitHub is used as the platform for managing and hosting the **source code** of the **Scientific Calculator** project. It provides **version control** capabilities using **Git** and allows developers to **track changes**, maintain **code history**, and collaborate efficiently.

In this project, the complete **source code** of the calculator application along with the configuration files such as the **Jenkins pipeline script**, **Dockerfile**, and **Ansible playbook** are stored in a **GitHub repository**. Whenever new changes are pushed to the repository, **Jenkins** is automatically triggered through a **webhook**, which starts the **CI/CD pipeline**.

GitHub therefore acts as the **central repository** that connects the **development stage** with the **automated build, testing, and deployment stages** of the pipeline.

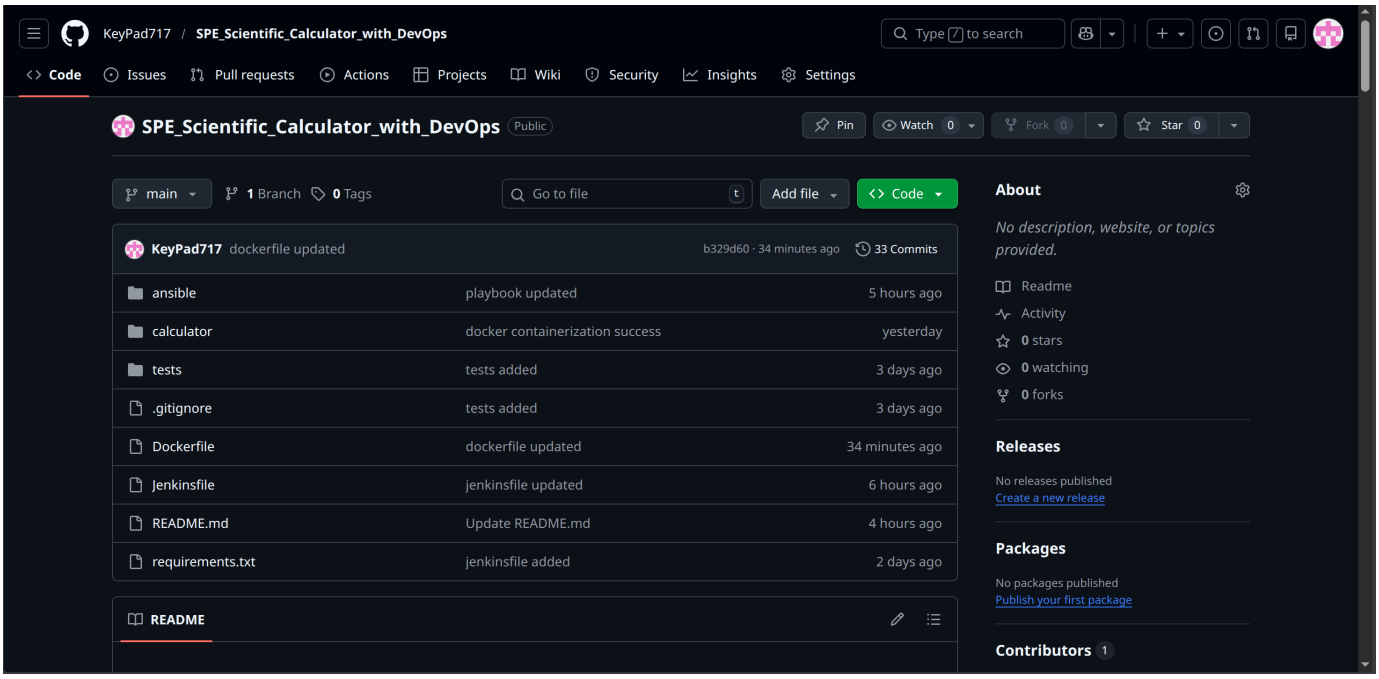



Figure 1: Github Repository

# 3 Project Workflow

The project follows a standard **CI/CD** workflow that automates software delivery from development to deployment.

The process begins when a developer pushes code changes to the **GitHub** repository. **GitHub** triggers **Jenkins** through a configured **webhook**, which starts executing the pipeline automatically without requiring manual intervention.

During pipeline execution, **Jenkins** retrieves the latest version of the source code from the repository. After fetching the source code, the pipeline installs the required dependencies and prepares the environment for execution.

Automated tests are then executed using the **Pytest** framework. The test suite verifies the correctness of all the calculator operations. Running automated tests ensures that the application logic works correctly before moving to the build stage.

If the tests pass successfully, **Jenkins** builds a **Docker** image for the application. Containerization ensures that the application runs reliably across different systems without dependency conflicts.

The **Docker** image is then pushed to **DockerHub** so that it can be easily accessed by deployment systems. Finally, **Ansible** is used to pull the latest **Docker** image and start the container automatically.

Figure 2: Jenkins Pipeline Stage View

# 4 Jenkins Pipeline Overview

**Jenkins** plays a central role in automating the **CI/CD** process for this project. The pipeline is defined using a **Jenkinsfile** stored in the **GitHub** repository.

Whenever code changes are pushed to **GitHub**, a **webhook** notification triggers **Jenkins**. Since Jenkins was running on a local machine, the **ngrok** tunneling tool was used to expose the Jenkins server to the internet so that GitHub could send webhook requests. Jenkins then automatically starts the pipeline and executes the defined stages.

The pipeline retrieves the latest source code, installs the required dependencies, and runs automated tests using **Pytest** to verify that the application behaves correctly.

If the tests pass, Jenkins builds a **Docker** image using a **Dockerfile** and pushes the image to **DockerHub**, which acts as a container registry.

Finally, **Ansible** deploys the application by pulling the latest Docker image from DockerHub and starting the container automatically.

# 5 Pipeline Stages Explanation

The **Jenkins** pipeline used in this project consists of multiple stages. Each stage performs a specific task that contributes to the automated delivery process.

## 5.1 Checkout Source Code

In this stage, **Jenkins** retrieves the latest version of the source code from the **GitHub** repository. This ensures that the pipeline always operates on the most recent code updates.

## 5.2 Install Dependencies

This stage installs all required dependencies needed for running the application and executing automated tests. Preparing the environment ensures that the application runs correctly in later stages.

## 5.3 Run Automated Tests

Automated tests are executed using the **Pytest** framework. The test suite validates the correctness of calculator operations such as addition, subtraction, multiplication, division, factorial, logarithm, power, and square root. Running these tests during the pipeline helps detect errors early and prevents faulty code from being deployed.

## 5.4 Build Docker Image

After successful testing, **Jenkins** builds a **Docker** image containing the application and its runtime environment using a **Dockerfile**. Containerization allows the application to run consistently across different machines without dependency issues.

## 5.5 Push Docker Image

The **Docker** image is then pushed to **DockerHub**. Storing the image in a registry allows it to be easily accessed for deployment by other systems.

## 5.6 Deploy with Ansible

In the final stage, **Ansible** automates the deployment of the application. The playbook stops and removes any existing container, pulls the latest **Docker** image from **DockerHub**, and starts a new container instance using the updated image.

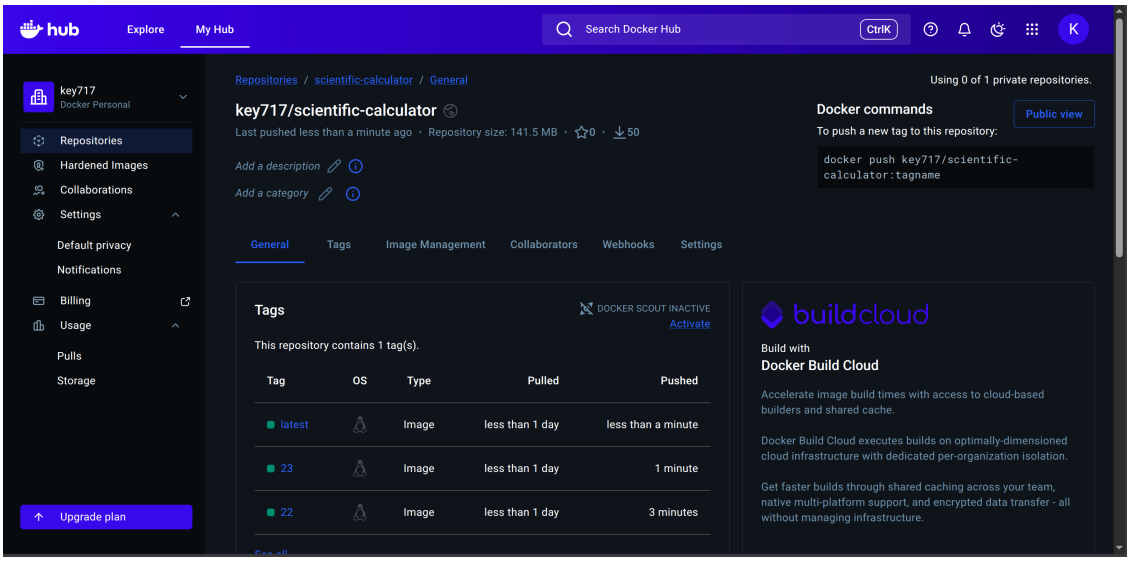



Figure 3: Docker Image Repository

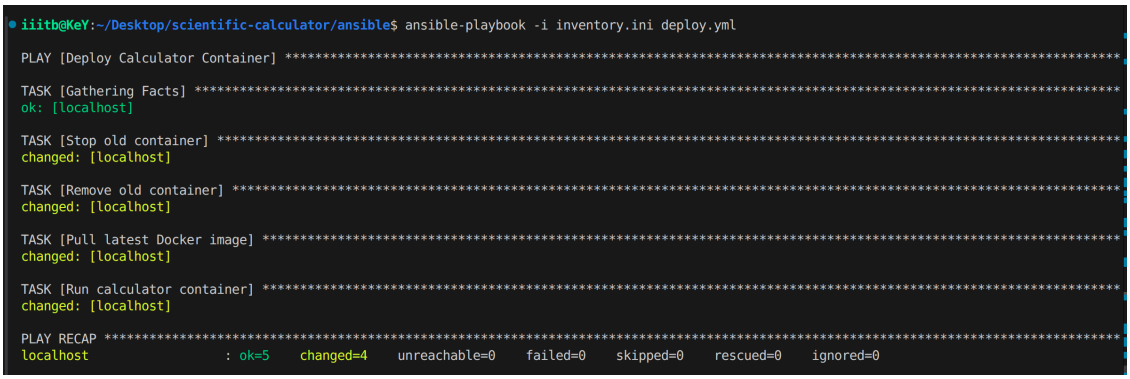



Figure 4: Ansible Playbook Execution

# 6 Automated Testing Results

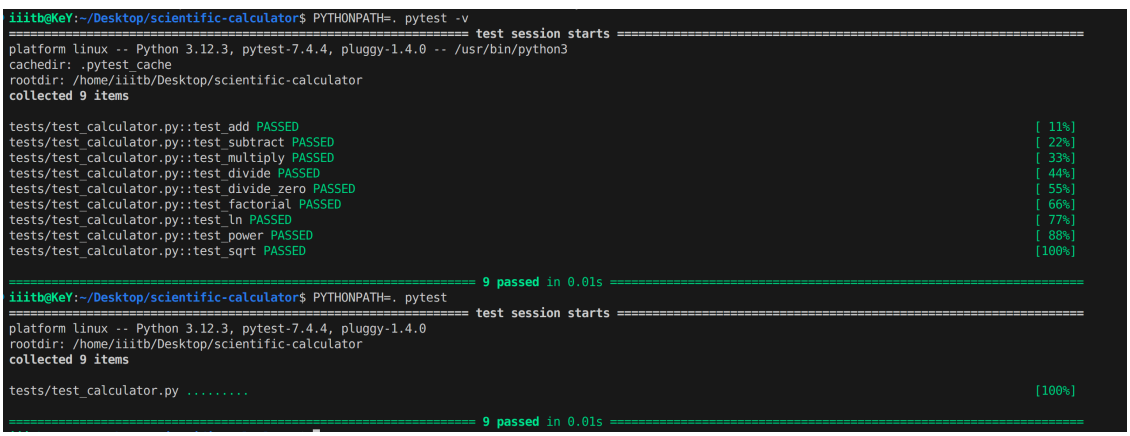
Automated testing is an important part of the **CI/CD** pipeline as it ensures that the application behaves correctly before deployment.

The scientific calculator application includes tests for various operations such as addition, subtraction, multiplication, division, factorial, natural logarithm, power, and square root.

These tests are implemented using the **Pytest** testing framework. **Jenkins** automatically runs these tests during the pipeline execution stage.

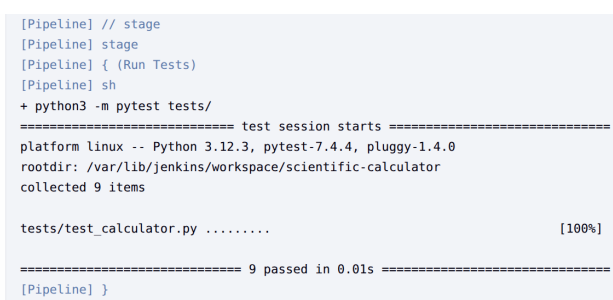
If all tests pass successfully, the pipeline proceeds to the next stages of building the **Docker** image and deployment. If any test fails, **Jenkins** stops the pipeline execution, preventing faulty code from being deployed.

Successful execution of the tests confirms that the application logic is functioning correctly and that the application is ready for deployment.



```
iiitb@KeY:~/Desktop/scientific-calculator$ PYTHONPATH=. pytest -v
============================================================ test session starts ============================================================
platform linux -- Python 3.12.3, pytest-7.4.4, pluggy-1.4.0 -- /usr/bin/python3
cachedir: .pytest_cache
rootdir: /home/iiitb/Desktop/scientific-calculator
collected 9 items

tests/test_calculator.py::test_add PASSED                                                                                                [ 11%]
tests/test_calculator.py::test_subtract PASSED                                                                                           [ 22%]
tests/test_calculator.py::test_multiply PASSED                                                                                           [ 33%]
tests/test_calculator.py::test_divide PASSED                                                                                             [ 44%]
tests/test_calculator.py::test_divide_zero PASSED                                                                                        [ 55%]
tests/test_calculator.py::test_factorial PASSED                                                                                          [ 66%]
tests/test_calculator.py::test_ln PASSED                                                                                                 [ 77%]
tests/test_calculator.py::test_power PASSED                                                                                              [ 88%]
tests/test_calculator.py::test_sqrt PASSED                                                                                               [100%]

============================================================= 9 passed in 0.01s =============================================================
iiitb@KeY:~/Desktop/scientific-calculator$ PYTHONPATH=. pytest
============================================================ test session starts ============================================================
platform linux -- Python 3.12.3, pytest-7.4.4, pluggy-1.4.0
rootdir: /home/iiitb/Desktop/scientific-calculator
collected 9 items

tests/test_calculator.py .........                                                                                                       [100%]

============================================================= 9 passed in 0.01s =============================================================
```

Figure 5: Automated Test Results



```
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Run Tests)
[Pipeline] sh
+ python3 -m pytest tests/
============================= test session starts ==============================
platform linux -- Python 3.12.3, pytest-7.4.4, pluggy-1.4.0
rootdir: /var/lib/jenkins/workspace/scientific-calculator
collected 9 items

tests/test_calculator.py .........                                      [100%]

============================== 9 passed in 0.01s ===============================
[Pipeline] }
```

Figure 6: Automated Test Results in Jenkins

# 7 Deployment Result

After the **Docker** image is pushed to **DockerHub**, the deployment process is handled automatically using **Ansible**.

The **Ansible** playbook pulls the latest **Docker** image from **DockerHub** and manages the container lifecycle. It stops any previously running container, removes the old container instance, pulls the latest image from **DockerHub**, and starts a new container using the updated image.

Containerization ensures that the application runs consistently regardless of the underlying system configuration.

The deployment can be verified using the **Docker** command *docker ps* to check the running containers.

```
iiitb@KeY:~$ docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED         STATUS         PORTS     NAMES
daae4c4c4aff   key717/scientific-calculator:latest  "sh -c 'python -m ca…"   5 seconds ago   Up 4 seconds             scientific-calculator
```

Figure 7: Running Docker Container

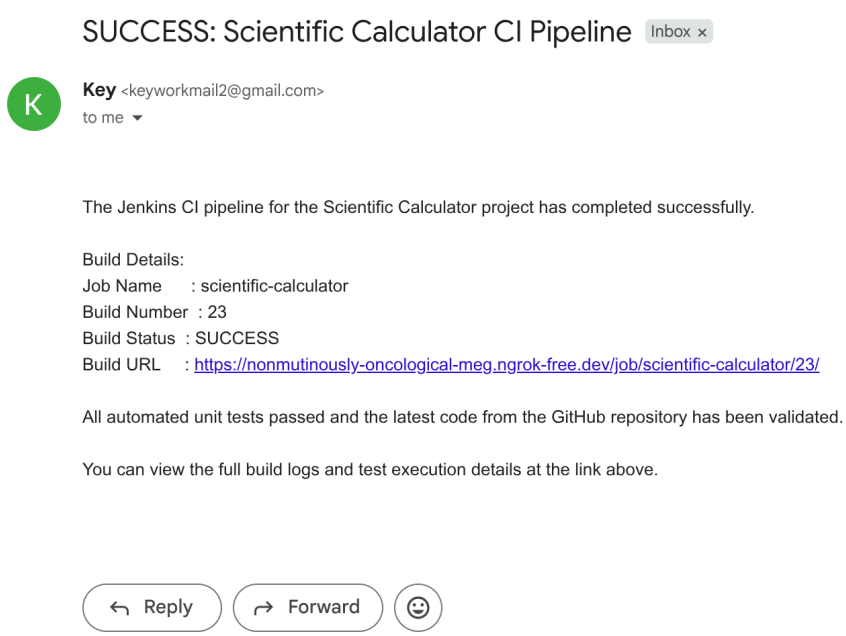



Figure 8: Email Notification

```
Scientific Calculator
1. add
2. subtract
3. multiply
4. divide
5. factorial
6. ln
7. power
8. sqrt
9. exit
Enter choice (1-9): 1
Enter first number: 1
Enter second number: 9
Result: 10.0

Scientific Calculator
1. add
2. subtract
3. multiply
4. divide
5. factorial
6. ln
7. power
8. sqrt
9. exit
Enter choice (1-9): 2
Enter first number: 4
Enter second number: 8
Result: -4.0
```

(a) Addition, Subtraction

```
Scientific Calculator
1. add
2. subtract
3. multiply
4. divide
5. factorial
6. ln
7. power
8. sqrt
9. exit
Enter choice (1-9): 3
Enter first number: 7
Enter second number: 2
Result: 14.0

Scientific Calculator
1. add
2. subtract
3. multiply
4. divide
5. factorial
6. ln
7. power
8. sqrt
9. exit
Enter choice (1-9): 4
Enter numerator: 8
Enter denominator: 0
Error
```

(b) Multiplication, Division

```
Scientific Calculator
1. add
2. subtract
3. multiply
4. divide
5. factorial
6. ln
7. power
8. sqrt
9. exit
Enter choice (1-9): 5
Enter integer (0-170): 3
Result: 6.0

Scientific Calculator
1. add
2. subtract
3. multiply
4. divide
5. factorial
6. ln
7. power
8. sqrt
9. exit
Enter choice (1-9): 6
Enter number: 5
Result: 1.6094379124341003
```

(c) Factorial, Natural Logarithm

```
Scientific Calculator
1. add
2. subtract
3. multiply
4. divide
5. factorial
6. ln
7. power
8. sqrt
9. exit
Enter choice (1-9): 7
Enter base: 4
Enter exponent: 3
Result: 64.0

Scientific Calculator
1. add
2. subtract
3. multiply
4. divide
5. factorial
6. ln
7. power
8. sqrt
9. exit
Enter choice (1-9): 8
Enter number: 64
Result: 8.0
```

(d) Power, Square Root

Figure 9: Calculator Operations Demonstration

# 8 Conclusion

This project demonstrates the successful implementation of a **CI/CD** pipeline for a **Python**-based **Scientific Calculator** application using modern DevOps tools.

The integration of **GitHub**, **Jenkins**, **Docker**, **Pytest**, **DockerHub**, and **Ansible** enables automated software delivery from development to deployment. The pipeline automatically retrieves code changes, executes automated tests, builds **Docker** images, pushes them to **DockerHub**, and deploys the application using **Ansible**.

Automation through **CI/CD** pipelines improves software reliability, reduces manual effort, and enables faster delivery of applications.

Overall, this project highlights how DevOps practices can streamline the software development lifecycle and improve efficiency in modern software systems.

**GitHub Repository:**
`https://github.com/KeyPad717/SPE_Scientific_Calculator_with_DevOps.git`

**DockerHub Image:**
`https://hub.docker.com/r/key717/scientific-calculator`